

Lesson 1: JavaScript Basics and Functions

Intro to Programming — Code the Dream

Today's mentors: _____ and _____

Session Overview

By the end of this session, you will be able to:

- Describe the three steps of the problem-solving process
- Create variables and identify common data types
- Write a basic conditional statement
- Write and call a function
- Use `console.log()` to check your code

Explore

Mentor 1 leads this section.

The Three Steps of Problem-Solving

1. **Understand** — Rewrite the problem in your own words
2. **Plan** — What are the inputs? What is the output?
3. **Divide** — Break the problem into smaller steps

Don't jump straight into code. Plan first!

Check for Understanding

Which step comes first when you get a new coding problem?

- A) Start typing code right away
- B) Rewrite the problem in your own words
- C) Search for the answer online
- D) Ask a classmate to solve it

Variables

A variable is a **container that stores a value**.

```
let firstName = "Maria";    // text – can change later
const birthYear = 1995;    // number – cannot change
```

- `let` — the value **can be reassigned**
- `const` — the value **stays the same**

We avoid `var` in modern JavaScript.

Data Types — Text and Numbers

String — text, wrapped in quotes

```
let city = "Chicago";  
let greeting = 'Hello!';
```

Number — integers, decimals, and negatives

```
let age = 28;  
let price = 9.99;  
let debt = -50;
```

Data Types — Booleans and NaN

Boolean — only two possible values

```
let isStudent = true;  
let hasGraduated = false;
```

NaN — "Not a Number" — what you get from invalid math

```
let result = 4 * "hello";    // NaN
```


Check for Understanding

What data type is "42" ?

- A) Number — it looks like a number
- B) String — it has quotes around it
- C) Boolean — it is either true or false
- D) NaN — it cannot be used as a number

Conditionals

Conditionals let your code **make decisions**.

```
let temperature = 75;

if (temperature > 80) {
  console.log("It's hot!");
} else if (temperature > 60) {
  console.log("Nice weather!");
} else {
  console.log("It's cold!");
}
```

JavaScript checks conditions **top to bottom** and runs the **first** block that is `true`.

Check for Understanding

What does this code print?

```
let score = 85;
if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else {
  console.log("C");
}
```

A) A B) B C) C D) Nothing — there's an error

Functions

A function is a **reusable block of code**.

```
function greet() {  
    return "Welcome to Code the Dream!";  
}  
  
console.log(greet());  
// "Welcome to Code the Dream!"
```

Write it once — use it as many times as you need.

Functions with Parameters

Parameters let you **pass information into** a function.

```
function greetStudent(name) {  
    return "Hello, " + name + "!";  
}  
  
console.log(greetStudent("Maria")); // "Hello, Maria!"  
console.log(greetStudent("James")); // "Hello, James!"
```

- `name` is the **parameter** — a placeholder inside the function
- `"Maria"` is the **argument** — the actual value passed in

Check for Understanding

What will this code print?

```
function double(num) {  
    return num * 2;  
}  
console.log(double(5));
```

A) 5 B) 10 C) "10" D) Nothing — return doesn't print

Naming Conventions — camelCase

JavaScript uses **camelCase** for variables and functions.

```
// Descriptive and readable  
let totalPrice = 29.99;  
let firstName = "Maria";  
function calculateSum(a, b) { return a + b; }
```

```
// Hard to read  
let tp = 29.99;  
let fn = "Maria";  
function cs(a, b) { return a + b; }
```

Good names make your code easier to read — and easier to debug.

Debugging Basics

When something goes wrong, use `console.log()` to investigate.

```
let firstName = "Maria";  
let lastName = "Santos";  
let fullName = firstName + lastName;  
  
console.log("Full name:", fullName);  
// Output: "Full name: MariaSantos" – missing space!
```

Common error types: syntax, reference, type, and logical.

Bugs are normal. Every developer debugs constantly.

Apply

Mentor 2 takes over here.

Core Activity: Let's Code Together

We'll build something step by step — **tell me what to type!**

Open your browser console:

- **Chrome / Edge:** Right-click → Inspect → Console
- **Firefox:** Right-click → Inspect → Console

Step 1 — Create Variables

Let's introduce a fictional class member.

```
let firstName = "Amara";  
let lastName = "Osei";  
let country = "Ghana";
```

Ask the group: How do we combine these into one sentence?

Step 2 — String Concatenation

Using `+`

```
let intro = "Hi! I'm " + firstName + " " + lastName  
            + " from " + country + ".";
```

Using template literals

```
let intro = `Hi! I'm ${firstName} ${lastName} from ${country}.`;
```

Which do you find easier to read?

Step 3 — Write a Function

Challenge: Write a function that greets someone by name.

```
function greetStudent(name) {  
    return "Welcome, " + name + "!";  
}  
  
console.log(greetStudent("Amara"));  
console.log(greetStudent("James"));
```

Follow-up: What if we want the name in all capitals?

Step 4 — Find the Bug

This code has a bug. Can you spot it?

```
function combineNames(first, last) {  
    return first + last;  
}  
  
console.log(combineNames("Amara", "Osei"));  
// Output: "AmaraOsei" – something is missing!
```

What would you change?

Step 4 — Bug Fixed

```
function combineNames(first, last) {  
    return first + " " + last;  
}  
  
console.log(combineNames("Amara", "Osei"));  
// Output: "Amara Osei"
```

Adding a " " between variables is one of the most common fixes in JavaScript.

Extension Activity

Only if students are ready and time allows.

Extension — Magic 8 Ball

Build a function that returns a random response.

```
function magic8Ball() {  
  let num = Math.floor(Math.random() * 3) + 1;  
  if (num === 1) return "It is certain.";   
  if (num === 2) return "Perhaps.";   
  return "Absolutely not.";   
}  
console.log(magic8Ball());
```

Try it: Call the function 5 times. Do you get different answers?

Assignment Preview

15 questions this week:

Questions	Topic
Q1–Q3	Variables (strings, numbers, booleans)
Q4–Q5	String concatenation and math
Q6	String methods
Q7–Q9	Conditionals
Q10–Q11	Number methods and type conversion
Q12–Q15	Writing functions

Also: Create your first **GitHub repository!**

Tips for the Assignment

1. **Read each question carefully** before writing any code
2. **Uncomment the `console.log()` lines** — remove the `//` to activate them
3. **Run your code often** — check output as you go, not just at the end
4. **Stuck?** Ask an AI for hints, not answers:

"Can you give me 3 hints without showing me the solution?"

Your GitHub Task This Week

Create a repository to store your code:

- [] GitHub → Repositories → click **New**
- [] Name it: `yourname-classname` (e.g. `maria-santos-jupiter`)
- [] Set to **Public**, check "Add a README file"
- [] Click **Create Repository**
- [] Paste the link in the "second link" field when submitting

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

How comfortable do you feel with variables and `console.log()` ?

Before next session:

- Complete the **lesson materials** (Odin Project or Scrimba)
- Finish the **15-question assignment**
- Create your **GitHub repository**
- Post questions in the **Slack** `#discussion` channel anytime

Resources

Resource	What it's for
Odin Project Fundamentals 1–3	Written lessons
Scrimba JS Deep Dive	Video lessons
MDN JavaScript Basics	Reference
W3Schools JavaScript	Quick lookups
Slack <code>#discussion</code> channel	Questions between sessions

Next week: Loops and Arrays!

Great work today!

You've taken your first step into programming.

Questions? Reach out on Slack anytime.